

W10 Pre-lab Quiz: Inter-process Communication

Started: Mar 6 at 5:50pm

Quiz Instructions



Week 10

Pre-Lab Quiz

Read all the study material for the module.

Read this article summarizing inter-process communication:

- [FreeRTOS – Part 4: Inter-Process Communication](https://circuitcellar.com/research-design-hub/design-solutions/freertos-part-4-inter-process-communication/) (https://circuitcellar.com/research-design-hub/design-solutions/freertos-part-4-inter-process-communication/)

Read the following Sections in the FreeRTOS documentation:

- [RTOS Software Timers](https://www.freertos.org/RTOS-software-timer.html) (https://www.freertos.org/RTOS-software-timer.html)
- [RTOS Task Notifications](https://www.freertos.org/RTOS-task-notifications.html) (https://www.freertos.org/RTOS-task-notifications.html)



Question 1 0.5 pts

Which of the following real-world examples would you probably **not** want to use a software timer for in an RTOS?



Turning off an LED after a timeout period



Controlling the precise timing of a stepper motor in a motion control system



Retrying a failed network connection after a delay



Timing a user inactivity period before entering sleep mode



Sending a periodic heartbeat signal to a monitoring system



Question 2 0.5 pts

Which of the following is true about using a task notification in an RTOS?



It is faster and uses less memory than other synchronization mechanisms like semaphores

☐

It automatically creates a queue for message passing

☐

It can be used to notify all tasks at once

☐

It requires dynamic memory allocation to function

☐

It must be used only from interrupt context



Question 3 0.5 pts

In STM32 microcontrollers, as with most microcontrollers, all memory space is accessible to all tasks, even when using FreeRTOS.

☐

True

☒

False



Question 4 0.5 pts

Which of the following is **not true** about using a task notification in an RTOS?

☒

Task notifications can be sent from both tasks and ISRs

☐

Task notifications can be used for simple signaling or data passing

☐

Task notifications can be used to notify multiple tasks at once

☐

Each task has a built-in notification value

☐

Task notifications are faster and use less memory than semaphores



Question 5 1 pts

What is the purpose of the xEventGroupClearBits() function?

☐

To delete the event group.

☐

To reset the event group to its initial state.

☐

To suspend all tasks using the event group.



To block a task until bits are cleared.



To clear specific bits in an event group.



Question 6 1 pts

What is the effect of setting xClearOnExit to pdTRUE in xEventGroupWaitBits()?



The task will yield to the idle task.



The task will be suspended after the wait.



The event group will be deleted after the wait.



The bits being waited on will be cleared automatically when the function returns.



The event group will be reset to zero.



Question 7 1 pts

Which of the following best describes a key advantage of using task notifications over other interprocess communication mechanisms in FreeRTOS?



They support queuing of multiple values like a message buffer.



They allow multiple tasks to wait on the same notification simultaneously.



They automatically reset after being received, unlike semaphores.



They are the most memory-efficient and fastest form of task-to-task signaling.



They are the only mechanism that can be used from both tasks and ISRs.



Question 8 1 pts

What is the effect of calling xTaskNotify() with eSetBits?



It clears all bits in the notification value.



It sets specific bits in the task's notification value without affecting other bits.

☐

It increments the notification value.

☐

It resets the task's priority.

☐

It overwrites the entire notification value.



Question 9 1 pts

What happens if a task calls `xTaskNotifyWait()` with `ulBitsToClearOnEntry = 0` and `ulBitsToClearOnExit = 0`?

☐

The task will block indefinitely.

☐

The task will reset its notification value.

☐

The task will yield to the idle task.

☐

The task will clear all bits before waiting.

☒

The task will not clear any bits before or after waiting.



Question 10 1 pts

Which of the following is true about task notifications and ISRs?

☐

ISRs must use queues instead of notifications.

☐

Notifications from ISRs are delayed until the task runs.

☒

ISRs can safely send task notifications using `xTaskNotifyFromISR()` or `vTaskNotifyGiveFromISR()`.

☐

Notifications from ISRs require disabling interrupts.

☐

ISRs cannot interact with task notifications.



Question 11 1 pts

Which of the following is true about `xTaskNotifyWait()` compared to `ulTaskNotifyTake()`?

☐

It is only usable from ISRs.



It cannot be used with a timeout.



It clears the notification value automatically.



It allows waiting for specific bits to be set, rather than just a count.



It is slower and uses more memory.



Question 12 1 pts

Which of the following is a key difference between `xTaskNotify()` and `xTaskNotifyGive()`?



`xTaskNotify()` is only usable from ISRs.



`xTaskNotify()` is slower and uses more memory.



`xTaskNotify()` allows sending a value or setting bits, while `xTaskNotifyGive()` only increments a counter.



`xTaskNotifyGive()` can notify multiple tasks at once.



`xTaskNotifyGive()` clears all bits in the notification value.

Quiz saved at 10:27am

Submit Quiz